

Solutionnaire du TP3

0. Charger le fichier

```
fruit <- read.table(file = "/home/public/R/fruit.tsv", sep = "\t", header = TRUE)
```

```
fruit
#>      FRUIT COULEUR QUANTITE
#> 1   Fraise   Rouge       10
#> 2   Banane   Jaune        5
#> 3  Abricot  Orange        4
#> 4   ananas   Jaune        5
#> 5   pomme   Rouge        7
#> 6   Melon   Orange        1
#> 7    Kiwi    Vert         2
#> 8   Cerise   Rouge        3
#> 9   Citron   Jaune        9
#> 10  Citron   Vert         8
#> 11 Framboise Rouge        2
#> 12  Raisin   Rouge        1
#> 13   Prune   Rouge        1
#> 14   Poire   Vert         8
#> 15 Pesteque Rouge        7
#> 16   Orange Orange        3
```

1. Nombre de fruits différents de couleur rouge

On construit un vecteur logique (`TRUE` pour les lignes rouges), puis on compte les `TRUE` avec `sum()`. La valeur dans le fichier est `"Rouge"` avec une majuscule : `fruit$COULEUR == "rouge"` donnerait 0.

```
est_rouge <- fruit$COULEUR == "Rouge"
sum(est_rouge)
#> [1] 7
```

Variante : on extrait les lignes, puis on compte les rangées.

```
nrow(fruit[est_rouge, ])
#> [1] 7
```

2. Trier par quantité croissante

`sort()` trie un **vecteur** et renvoie les valeurs triées : on perdrait le lien avec les noms de fruits. `order()` renvoie les **positions** des lignes dans l'ordre croissant, qu'on utilise comme index de lignes.

```
sort(fruit$QUANTITE) # seulement les valeurs
#> [1] 1 1 1 2 2 3 3 4 5 5 7 7 8 8 9 10
order(fruit$QUANTITE) # les positions des lignes
#> [1] 6 12 13 7 11 8 16 3 2 4 5 15 10 14 9 1
fruit_trie <- fruit[order(fruit$QUANTITE), ]
fruit_trie
#>      FRUIT COULEUR QUANTITE
#> 6      Melon  Orange        1
#> 12     Raisin   Rouge        1
#> 13     Prune   Rouge        1
#> 7       Kiwi    Vert         2
#> 11 Frambroise  Rouge        2
#> 8       Cerise  Rouge        3
#> 16      Orange  Orange        3
#> 3      Abricot  Orange        4
#> 2       Banane  Jaune         5
#> 4      ananas  Jaune         5
#> 5       pomme  Rouge         7
#> 15   Pastèque  Rouge         7
#> 10      Citron  Vert          8
#> 14      Poire   Vert          8
#> 9       Citron  Jaune         9
#> 1       Fraise  Rouge        10
```

3. Les 3 fruits les moins abondants

```
fruit_trie[1:3, ]
#>      FRUIT COULEUR QUANTITE
#> 6      Melon  Orange        1
#> 12 Raisin   Rouge        1
#> 13 Prune    Rouge        1
```

Équivalent : `head(fruit_trie, 3)`. Remarque : plusieurs fruits ont la même quantité (1), l'ordre entre eux suit l'ordre du fichier.

4. Nombre total de fruits

```
sum(fruit$QUANTITE)
#> [1] 76
```

5. Nombre total de fruits rouges

On réutilise le vecteur logique de la question 1 pour ne sommer que les quantités des lignes rouges.

```
sum(fruit$QUANTITE[est_rouge])  
#> [1] 31
```

Et si... ?

1. Trier avec `sort()`

```
sort(fruit)  
#> Error in xtfm.data.frame(x): cannot xtfm data frames
```

`sort()` ne sait trier qu'un vecteur : sur un `data.frame`, R s'arrête avec une erreur.

```
fruit[sort(fruit$QUANTITE), ]  
#>      FRUIT COULEUR QUANTITE  
#> 1   Fraise   Rouge      10  
#> 1.1 Fraise   Rouge      10  
#> 1.2 Fraise   Rouge      10  
#> 2   Banane   Jaune       5  
#> 2.1 Banane   Jaune       5  
#> 3   Abricot Orange       4  
#> 3.1 Abricot Orange       4  
#> 4   ananas   Jaune       5  
#> 5    pomme   Rouge       7  
#> 5.1 pomme   Rouge       7  
#> 7     Kiwi    Vert       2  
#> 7.1 Kiwi     Vert       2  
#> 8    Cerise   Rouge       3  
#> 8.1 Cerise   Rouge       3  
#> 9    Citron   Jaune       9  
#> 10 Citron    Vert       8
```

Ici, **aucune erreur**, mais le résultat est faux : `sort()` renvoie les *valeurs* triées (1, 1, 1, 2, ...), que R utilise comme *numéros de rangées*. On obtient trois fois la rangée 1 (Fraise), puis des rangées sans rapport avec le tri. C'est le cas le plus dangereux : une erreur silencieuse ne se voit que si on vérifie le résultat.

2. Du plus abondant au moins abondant

```
head(fruit[order(fruit$QUANTITE, decreasing = TRUE), ], 3)  
#>      FRUIT COULEUR QUANTITE  
#> 1   Fraise   Rouge      10  
#> 9    Citron   Jaune       9  
#> 10 Citron    Vert       8
```

Équivalent : `fruit[order(-fruit$QUANTITE), ]`.

3. `=` au lieu de `==`

```
fruit[fruit$COULEUR = "Rouge", ]
```

Error: unexpected '=' in "fruit[fruit\$COULEUR ="

`=` sert à assigner une valeur (comme `<-`) ou à nommer un argument ; `==` sert à **comparer**. Ici, R refuse la ligne avant même de l'exécuter : c'est une erreur de syntaxe.

4. `"Rouge"` et `"rouge"` mélangés

`tolower()` met tout en minuscules avant la comparaison :

```
sum(tolower(fruit$COULEUR) == "rouge")  
#> [1] 7
```

5. Le fruit le plus abondant

```
max(fruit$QUANTITE)  
#> [1] 10  
fruit[fruit$QUANTITE == max(fruit$QUANTITE), "FRUIT"]  
#> [1] "Fraise"
```